

AutoCTM Software Design Specification

Abstract

This document is the Software Design Specification for AutoCTM, a Certificate Transparency monitor written in Go. It describes the system architecture, component responsibilities, inter-process communication, data schema, and error handling strategies that guide the implementation of AutoCTM. The system is composed of three decoupled processes (a CLI, a persistent broker daemon, and one or more independent monitor instances) that together enable operators to monitor RFC 9162-compliant CT logs for certificate issuance against a configurable domain watchlist

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Known Limitations	3
1.3.1	Broker as Single Point of Failure	3
1.4	Definitions	3
1.5	References	4
2	System Overview	4
2.1	Separation of Responsibilities	4
3	Architecture	4
3.1	Component Diagram	4
3.2	Process Lifecycle	4
3.2.1	Broker Startup	4
3.2.2	CLI Lifecycle	5
3.2.3	Instance Lifecycle	5
3.2.4	Broker Restart Behaviour	5
3.3	Inter-Process Communication	5
4	Component Design	5
4.1	CLI	5
4.1.1	Command Contexts	5
4.1.2	Auto-Spawn Behaviour	6
4.2	Broker	6
4.2.1	Responsibilities	6
4.3	Monitor Instance	7
4.3.1	Startup Sequence	7
4.3.2	Broker Connection Resilience	7
4.3.3	Polling Loop	7
4.3.4	System Shutdown	7
4.3.5	Watchlist Checking	7
4.3.6	CRL Checking	8
5	Data Design	8
5.1	Database	8
5.2	Schema	8
5.2.1	certificates	8
5.2.2	watchlist	8
5.2.3	instances	8
5.2.4	sth_state	9
5.2.5	alerts	9
6	Error Handling	9
6.1	Log Unavailability	9
6.2	Consistency Proof Failure	9
6.3	Database Connectivity	9
6.4	Broker Unavailability	9
6.5	Instance Crash	9

Version	Date	Author	Description
0.1	2025-03-24	Oscar Ashburn	Initial draft

Table 1: Revision History

1 Introduction

1.1 Purpose

This document describes the software design of AutoCTM, a Certificate Transparency monitor written in Go. It is intended to guide implementation and serve as a reference for the development team.

1.2 Scope

AutoCTM allows operators to monitor one or more RFC 9162-compliant CT logs for certificate issuance against a configurable domain watchlist. The system consists of three decoupled components: a CLI, a persistent broker daemon, and one or more independent monitor instances. The CLI may be closed at any time without affecting running instances. On relaunch the CLI reconnects to the broker and resumes control.

1.3 Known Limitations

1.3.1 Broker as Single Point of Failure

The broker is a single point of failure for the control plane. If the broker dies:

- The CLI cannot connect or issue commands.
- Instance status becomes unknown until the broker is restarted.

This is an accepted limitation for a small internal tool. Instances are designed to continue monitoring and writing to the database independently of the broker. Operators can restart the broker at any time to resume control without affecting running instances.

1.4 Definitions

- **CLI** — The operator-facing command-line interface. A short-lived process that connects to the broker, issues commands, and may be closed freely.
- **Broker** — A persistent background daemon that owns the control socket, manages the instance registry, and routes commands from the CLI to the appropriate instance. The broker is control plane only — it does not participate in certificate monitoring.
- **Instance** — A long-running background process responsible for monitoring one or more CT logs, checking certificates, and raising alerts. Instances load their configuration directly from the database and operate independently of the broker at runtime.
- **Active Instance** — The instance currently selected in the CLI via `/set-instance`. Commands issued in instance context are routed to this instance.
- **STH** — Signed Tree Head, as defined in RFC 9162.
- **SCT** — Signed Certificate Timestamp, as defined in RFC 9162.
- **MMD** — Maximum Merge Delay, as defined in RFC 9162.
- **Watchlist** — The set of domains configured by the operator for alert monitoring.

1.5 References

- RFC 9162 — Certificate Transparency Version 2.0
- AutoCTM Software Requirements Specification

2 System Overview

AutoCTM is composed of three decoupled processes:

- **CLI** (`autoctm`) — The operator entry point. On launch it checks for a running broker. If none is found it spawns one automatically before presenting the command prompt. The CLI may be closed at any time; running instances are unaffected.
- **Broker** (`autoctm-broker`) — A persistent daemon that starts automatically on first CLI launch. It owns a local control socket at a well-known path, maintains the instance registry, and routes commands between the CLI and instances. It remains alive independently of both the CLI and any instance. The broker is control plane only — loss of the broker does not affect instance monitoring.
- **Monitor Instance** (`autoctm-instance`) — A long-running background process spawned by the broker on `/start`. Each instance loads its configuration directly from the database, independently polls its configured CT logs, processes entries, performs watchlist and CRL checks, and raises alerts. Multiple instances may run concurrently. Instances continue to operate if the broker becomes unavailable.

2.1 Separation of Responsibilities

To avoid ambiguity over configuration ownership, the system follows a clear separation of responsibilities:

- **Database** — The single source of truth for all configuration and state, including instance config, log lists, watchlist, STH state, and alerts.
- **Broker** — Orchestration only. Spawns instances, maintains the instance registry, and routes control commands. Does not own configuration.
- **Instance** — Worker. Loads all configuration directly from the database on startup and re-reads it when notified of a change by the broker.

3 Architecture

3.1 Component Diagram

3.2 Process Lifecycle

3.2.1 Broker Startup

1. CLI launches and checks for a broker socket at the well-known path.
2. If no socket is found, the CLI forks and execs `autoctm-broker` as a detached background process, then waits for the socket to appear.
3. The broker creates the socket, loads the instance registry from the database, and begins listening for CLI and instance connections.

3.2.2 CLI Lifecycle

1. On launch, the CLI connects to the broker socket.
2. The CLI enters a REPL, accepting operator commands and forwarding them to the broker.
3. The CLI maintains an *active instance* context, initially unset (global context).
4. The CLI may be closed at any time with no effect on the broker or any running instance. On relaunch it reconnects to the existing broker.

3.2.3 Instance Lifecycle

1. The broker spawns a new instance process on receipt of `/start`, passing only the instance ID.
2. The instance connects to the database and loads its full configuration directly from the `instances` table.
3. The instance attempts to register with the broker via the control socket. If the broker is unavailable, the instance shall continue startup and retry registration periodically.
4. The instance spawns one goroutine per configured log and enters its polling loop.
5. On `/shutdown` the broker signals the instance to drain its current work and exit cleanly.
6. On exit, the instance updates its status to `stopped` in the database.

3.2.4 Broker Restart Behaviour

If the broker is restarted it shall:

1. Recreate the control socket at the well-known path.
2. Reload the instance registry from the database.
3. Wait for instances to reconnect and re-register. The broker shall not attempt to reconnect to instances — instances are responsible for detecting the lost connection and re-registering with the restarted broker.
4. Mark any instance that does not re-register within a configurable timeout as `stopped`.

3.3 Inter-Process Communication

All IPC uses a local control socket owned by the broker at a well-known path. Messages are encoded as newline-delimited JSON. There are two connection types:

- **CLI → Broker** — The CLI holds a persistent connection to the broker for the duration of its session.
- **Instance → Broker** — Each instance initiates and maintains a connection to the broker for registration and command receipt. If the broker connection is lost, the instance shall keep running and retry the connection periodically until the broker is available again.

4 Component Design

4.1 CLI

4.1.1 Command Contexts

The CLI operates in one of two contexts at any time:

Global Context When no active instance is selected, the CLI may execute system-level or read-only broker commands. The commands sent are to be in a **JSON Object**:

- `/start` — Instruct the broker to spawn a new monitor instance. The broker persists the instance in the database and returns the instance ID.
- `/list-instances` — List all registered instances and their status.
- `/set-instance <instanceID>` — Select an active instance for subsequent commands.
- `/exit` — Return to global context.

These operations are no longer CLI-responsible and are instead handled through broker-routed instance or database workflows.

Instance Context An active instance is selected via `/set-instance`. All instance-scoped commands are routed through the broker.

- `/add-log <url>` — Request broker to update instance configuration and notify the instance to reload.
- `/remove-log <url>` — Remove CT log from instance configuration via broker update.
- `/pause` — Broker signals instance to pause polling.
- `/resume` — Broker signals instance to resume polling.
- `/status` — Broker returns current instance status, including logs, last STH, and alert counts.
- `/shutdown` — Shut down the selected instance only via broker. instance only.
- `/exit` — Return CLI to global context.

4.1.2 Auto-Spawn Behaviour

If the CLI cannot connect to the broker socket on launch it shall automatically spawn the broker as a detached background process before presenting the prompt. This requires no manual daemon management from the operator.

4.2 Broker

4.2.1 Responsibilities

The broker is responsible for orchestration only. It shall:

- Own and maintain the local control socket.
- Use database registry of connected instances.
- Accept and route CLI commands to the correct instance.
- Spawn new instance processes on `/start`, writing the new instance record to the database before spawning.
- Accept re-registration from instances after a broker restart.
- Write instance status changes to the database.

The broker shall not hold instance configuration in memory. All configuration reads and writes go through the database.

4.3 Monitor Instance

4.3.1 Startup Sequence

1. Receive instance ID from the broker at spawn time.
2. Connect to PostgreSQL and load full configuration from the `instances` table using the instance ID.
3. Attempt to register with the broker. If unavailable, continue and retry in the background.
4. Spawn one goroutine per configured log.
5. Enter polling loop.

4.3.2 Broker Connection Resilience

Instances shall treat the broker connection as optional at runtime:

- If the broker connection is lost, the instance shall continue its polling loop uninterrupted.
- The instance shall periodically retry connecting and registering with the broker using an exponential backoff.
- On successful reconnection, the instance shall re-register and resume accepting commands.

4.3.3 Polling Loop

Each log goroutine shall independently perform the following steps:

1. Fetch the current STH via `GET /ct/v2/get-sth`.
2. Verify the STH signature using the log's public key.
3. If the STH is unchanged from the stored STH, wait for the configured poll interval and retry.
4. Fetch new entries since the last known STH via `GET /ct/v2/get-entries`.
5. For each entry, perform watchlist and CRL checks.
6. Verify the Merkle consistency proof between the previous and new STH via `GET /ct/v2/get-sth-consistency`.
7. Persist new entries and update the stored STH in the database.
8. Dispatch any pending alerts via email.
9. Wait for the configured poll interval and return to step 1.

4.3.4 System Shutdown

System-wide shutdown is no longer initiated from CLI commands directly.

Instead, shutdown behavior is delegated to the broker, which is responsible for coordinating termination of all instances and itself upon receiving a broker-level shutdown signal (external signal or administrative action).

4.3.5 Watchlist Checking

Each certificate's Subject CN and SAN fields are matched against all domains in the watchlist. Matching shall support:

- Exact domain match (e.g. `example.com`).
- Wildcard SAN match (e.g. `*.example.com`).

A match shall produce an alert record of type `watchlist.match`.

4.3.6 CRL Checking

The instance shall maintain a local in-memory cache of CRLs keyed by issuer DN. CRLs shall be refreshed on a configurable interval. Each observed certificate serial number shall be checked against the cached CRL for its issuer. A hit shall produce an alert record of type **revoked**.

5 Data Design

5.1 Database

AutoCTM uses a local PostgreSQL instance as its storage backend. The database is the single source of truth for all configuration and state. It is shared across the broker and all monitor instances.

5.2 Schema

5.2.1 certificates

Stores all observed certificate entries.

- **id** — Primary key (UUID).
- **leaf_hash** — SHA-256 leaf hash (unique).
- **log_id** — OID of the source log.
- **observed_at** — Timestamp of observation.
- **not_before**, **not_after** — Certificate validity period.
- **subject** — Certificate subject CN.
- **sans** — Array of Subject Alternative Names.
- **issuer** — Issuer DN.
- **serial** — Certificate serial number.
- **revoked** — Boolean, updated by CRL checks.

5.2.2 watchlist

- **id** — Primary key (UUID).
- **domain** — Domain pattern (exact or wildcard).
- **added_at** — Timestamp of addition.

5.2.3 instances

Stores instance identity, status, and configuration. This is the source of truth for instance configuration — instances load directly from this table on startup.

- **Name** — Name of instance.
- **id** — Instance ID (UUID).
- **pid** — OS process ID.
- **status** — Current status (**running**, **paused**, **stopped**).
- **started_at** — Timestamp.
- **config** — JSONB blob of instance configuration (log URLs, public keys, poll interval). Written by the broker, read by the instance.

5.2.4 sth_state

Tracks the last known STH per log per instance.

- `instance_id` — Foreign key to `instances`.
- `log_id` — OID of the log.
- `tree_size` — Tree size of the last known STH.
- `root_hash` — Root hash of the last known STH.
- `timestamp` — STH timestamp.

5.2.5 alerts

- `id` — Primary key (UUID).
- `certificate_id` — Foreign key to `certificates`.
- `instance_id` — Foreign key to `instances`.
- `reason` — Alert reason (`watchlist_match`, `revoked`, `log_misbehaviour`).
- `detail` — JSONB blob of alert-specific detail.
- `raised_at` — Timestamp.
- `dispatched` — Boolean, whether the notification was sent.

6 Error Handling

6.1 Log Unavailability

If a CT log is unreachable the instance shall log the failure and back off using exponential retry. If the log remains unavailable beyond the MMD the instance shall raise a `log_misbehaviour` alert.

6.2 Consistency Proof Failure

A consistency proof verification failure shall be treated as log misbehaviour. The instance shall record the failure, raise an alert, and pause polling of the affected log pending operator review via `/resume`.

6.3 Database Connectivity

If the database becomes unreachable the instance shall pause processing and retry on a backoff schedule. On recovery the instance shall resume from the last persisted STH state, re-fetching any entries missed during the outage.

6.4 Broker Unavailability

If the CLI cannot reach the broker it shall attempt to auto-spawn a new broker instance. If the broker cannot be spawned the CLI shall print a diagnostic error and exit.

If a running instance loses its connection to the broker it shall continue monitoring uninterrupted and retry registration on an exponential backoff until the broker is available again.

6.5 Instance Crash

If the instance crashes without a clean exit, the broker shall detect the lost connection and mark the instance as `stopped` in the database.

If the broker detects that an instance connection has been lost unexpectedly, it shall mark the instance as `stopped` in the database.